

GPU Infrastructure 101

**A Practical Guide
for DevOps, Cloud,
Platform and SRE
Engineers**



GPU Fundamentals



Linux & Drivers



Cloud & Infrastructure



Docker & Kubernetes



Monitoring & Troubleshooting



Security & Cost Management



Hands-On Practice



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

01

GPU Infrastructure Explained

1. WHAT IS GPU INFRASTRUCTURE?

GPU infrastructure is the combination of hardware, software, networking, storage and management tools used to run workloads that require high parallel computing power.

It includes physical servers with GPUs, cloud GPU instances and the supporting systems needed to operate, secure, monitor and scale GPU workloads.

2. WHY DOES GPU INFRASTRUCTURE EXIST?

- Modern workloads such as AI, machine learning, data analytics and rendering require massive parallel processing.
- CPUs are general purpose, but GPUs can perform thousands of tasks at the same time.
- GPU infrastructure exists to provide the compute power, memory, and specialized environment needed for these workloads.

3. WHERE GPU INFRASTRUCTURE FITS IN A REAL ENVIRONMENT



4. KEY COMPONENTS

- GPU Hardware**
GPUs with high compute cores and high memory (VRAM).
- Drivers & Software**
GPU drivers, CUDA, container runtime and management tools.
- Compute Resources**
GPU servers or cloud GPU instances.
- Storage**
High-performance storage for large datasets.
- Networking**
High bandwidth and low latency for distributed workloads.
- Monitoring & Management**
Tools to track GPU utilization, temperature, memory and cost.

5. COMMON USE CASES

- AI / Machine Learning**
 - Model training
 - Model inference
 - Large language models
- Rendering & Media**
 - 3D rendering
 - Video processing
 - Game development
- Data Analytics**
 - Large scale data processing
 - Data science workloads
 - ETL with GPU acceleration
- Scientific Computing**
 - Simulations
 - Research workloads
 - High performance computing

6. DEVOPS CONNECTION

- Provision and scale GPU infrastructure.
- Automate deployments using containers and Kubernetes.
- Monitor GPU usage, health and cost.
- Ensure security, access control and efficient resource usage.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

02 CPU vs GPU

How CPUs and GPUs differ, parallel processing, and why AI workloads benefit from GPUs.

1. WHAT ARE CPU AND GPU?

CPU Central Processing Unit



- General purpose processor
- Few powerful cores (usually 4-64 cores)
- Excellent for complex tasks and decision making
- Used for operating systems, applications and general workloads

GPU Graphics Processing Unit



- Specialized processor
- Thousands of smaller cores (1,000s of cores)
- Designed for parallel processing
- Excellent for performing the same operation on large amounts of data
- Used for AI, machine learning, rendering and scientific computing

2. CORE ARCHITECTURE DIFFERENCE

CPU (Optimized for Complex Tasks)



Large Cache

Control Logic

- Fewer, larger cores
- Each core is very powerful
- Handles complex and sequential tasks well

GPU (Optimized for Parallel Tasks)



High Bandwidth Memory (VRAM)

Control Logic

- Thousands of smaller cores
- Designed for parallel processing
- Handles many tasks at the same time

3. HOW THEY PROCESS WORK

CPU – SEQUENTIAL PROCESSING



- CPU processes tasks one at a time or a few at a time.
- Great for tasks that require complex logic or different operations.

GPU – PARALLEL PROCESSING



- GPU breaks a large task into many small tasks and processes them at the same time.
- Ideal for workloads where the same operation is performed on large amounts of data.

4. REAL WORLD ANALOGY

CPU A skilled chef in a restaurant



- Handles complex and different tasks
- High skill and decision making
- Slower when many simple dishes need to be prepared

GPU A large team of chefs



- Many chefs doing the same task
- Ideal for preparing many similar dishes at the same time
- Much faster for large quantities of similar work

5. WHY AI WORKLOADS BENEFIT FROM GPUS

- 1 AI models process massive amounts of data using matrix calculations.
- 2 These calculations are the same type of operation repeated many times, which is perfect for GPUs.
- 3 GPUs can perform thousands of calculations in parallel, making training and inference much faster.
- 4 GPUs also have high memory bandwidth (VRAM), which is essential for large AI models.
- 5 This reduces the time to train models and improves performance for real-time inference.



KEY TAKEAWAY

CPUs are built for complex, sequential tasks. GPUs are built for parallel processing, which makes them ideal for AI, machine learning and other data-intensive workloads.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

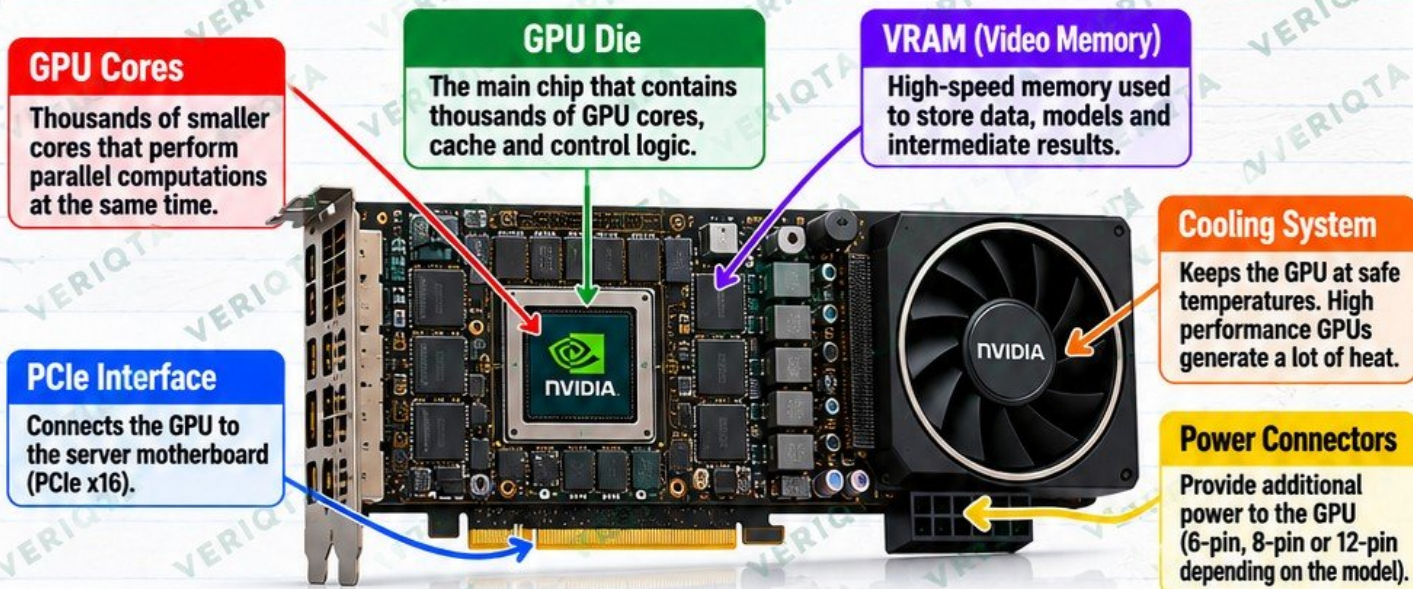


@veriqta

03 Inside a GPU

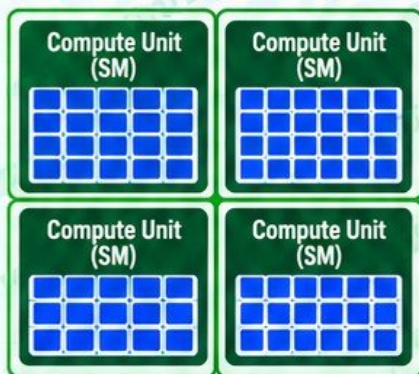
GPU cores, memory, compute units and the basic hardware components engineers should recognize.

1. KEY GPU HARDWARE COMPONENTS



2. HOW GPU CORES ARE ORGANIZED

GPU (Simplified View)



- = GPU Core (small, simple core)
- = Compute Unit / Streaming Multiprocessor (SM)

Multiple Small Cores

- A GPU has thousands of small, efficient cores.
- These cores are grouped into Compute Units (Streaming Multiprocessors on NVIDIA GPUs).

Designed for Parallel Work

- Each core can handle a small part of a large task.
- This allows the GPU to process many operations at the same time.

3. IMPORTANT GPU COMPONENTS



GPU Cores

Thousands of cores for parallel processing.



Compute Units (SM)

Group of cores, cache and scheduling logic.



VRAM (Video Memory)

High-speed memory for data, models and results.



Cache

Stores frequently used data for faster access.



Control Logic

Manages scheduling, memory access and communication between components.



Power System

Supplies enough power for high-performance computing.



Cooling System

Removes heat to keep the GPU stable and performing well.



PCIe Interface

Connects the GPU to the server for data transfer.

4. REAL EXAMPLE: NVIDIA GPU (SIMPLIFIED)

Component	Example (NVIDIA A100)
GPU Cores	6,912 CUDA cores
Compute Units (SM)	108 SMs
VRAM	40 GB HBM2
Memory Bandwidth	1,555 GB/s
PCIe Interface	PCIe 4.0 x16
Power (TDP)	400W
Cooling	Active cooling (fans)

Why this matters?

- More cores = higher parallel processing power.
- More VRAM = larger models and datasets.
- Higher memory bandwidth = faster data processing.
- Proper cooling and power = stable performance in production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

04 Understanding GPU Memory

VRAM, memory capacity, bandwidth, allocation and out-of-memory failures.

1. WHAT IS GPU MEMORY?

- GPU memory is called **VRAM (Video Random Access Memory)**.
- VRAM stores the data that the GPU uses for fast parallel processing.
- It holds models, datasets, textures, intermediate results and more.
- GPU memory is separate from system RAM and is much faster.

2. GPU MEMORY VS SYSTEM MEMORY

Feature	GPU Memory (VRAM)	System Memory (RAM)
Purpose	Stores data for GPU computations	Used by CPU and OS
Location	On the GPU	On the server motherboard
Speed	Very high bandwidth	Lower bandwidth
Size	Typically 8 GB – 80 GB+	Typically 16 GB – 1 TB+
Access	Used by GPU directly	Used by CPU and other components
Use cases	Models, datasets, intermediate results	OS, applications, general data

3. HOW GPU MEMORY IS USED



4. KEY GPU MEMORY TERMS

- VRAM Capacity**
Total amount of memory on the GPU (e.g., 8 GB, 24 GB, 40 GB, 80 GB).
- Memory Usage**
Amount of VRAM currently in use.
- Memory Bandwidth**
How fast data can be read from or written to VRAM (e.g., 500 GB/s, 1 TB/s).
- Memory Allocation**
Reserving VRAM for models, tensors and workloads.
- Memory Fragmentation**
VRAM becomes split into small unused blocks, which can cause allocation failures.

5. VRAM CAPACITY EXAMPLES

GPU Model	VRAM	Memory Bandwidth
NVIDIA T4	16 GB GDDR6	320 GB/s
NVIDIA L4	24 GB GDDR6	300 GB/s
NVIDIA A10	24 GB GDDR6	600 GB/s
NVIDIA A100	40 GB HBM2	1,555 GB/s
NVIDIA H100	80 GB HBM3	3,350 GB/s

6. WHAT CAUSES OUT-OF-MEMORY (OOM) ERRORS?

- Model is larger than available VRAM.
- Batch size is too large.
- Multiple processes are using VRAM.
- Memory fragmentation.
- Long-running workloads without releasing memory.
- VRAM leak in application or framework.
- Not enough free VRAM for a new allocation.

7. EXAMPLE: GPU MEMORY USAGE

```
$ nvidia-smi
```

GPU	Name	Memory-Usage
0	NVIDIA A100	1024MiB / 40960MiB

1 GB currently used 40 GB total VRAM

8. HOW TO FIX GPU MEMORY ISSUES

- Use a smaller model or batch size.
- Close unnecessary GPU processes.
- Release unused memory in your application.
- Use model quantization (smaller model size).
- Use GPU with larger VRAM if needed.
- Restart the workload or server if memory is not released.
- Monitor memory usage regularly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

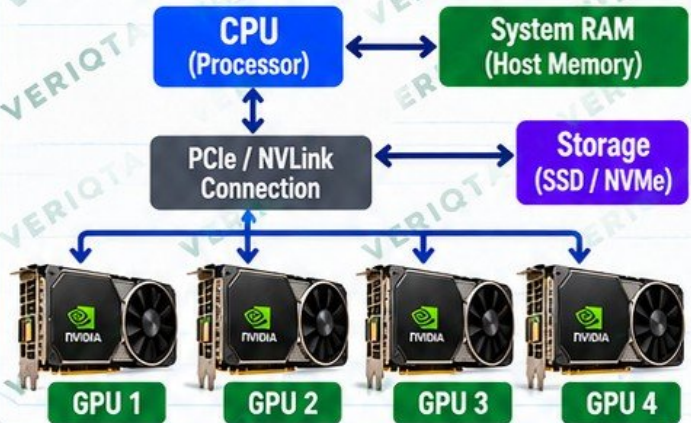
05 How GPUs Connect to Servers

PCIe, NVLink, NVSwitch and the importance of hardware connectivity.

1. OVERVIEW

- GPUs connect to servers using high-speed links that allow data to move between the GPU, CPU, memory and other GPUs.
- The main connection technologies are PCIe, NVLink and NVSwitch.
- The type of connection and bandwidth affect GPU performance, especially for large AI workloads.

2. SERVER WITH GPU (SIMPLIFIED ARCHITECTURE)

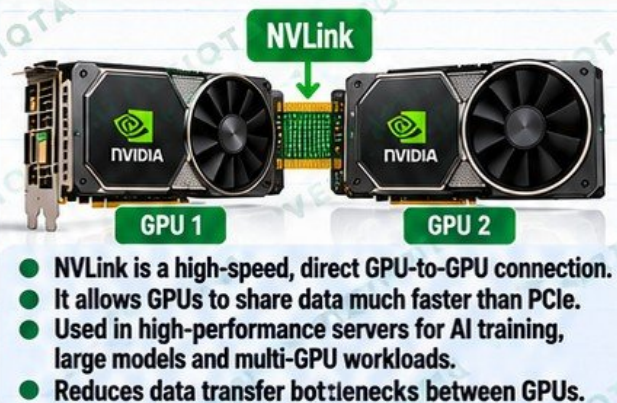


3. PCIe (PERIPHERAL COMPONENT INTERCONNECT EXPRESS)



- PCIe is the most common connection between GPUs and the server motherboard.
- Modern GPUs use PCIe Gen4 or Gen5 for higher bandwidth.
- A single GPU typically uses a PCIe x16 slot.
- PCIe provides high bandwidth but is mainly a connection between the CPU and each GPU.

4. NVLINK (NVIDIA HIGH-SPEED LINK)



5. NVSWITCH (GPU-TO-GPU SWITCH)



- NVSwitch is a high-speed switch that connects many GPUs together.
- It allows any GPU to communicate with any other GPU at high speed.
- Common in large AI servers with 4, 8 or more GPUs.
- Provides much higher total bandwidth than PCIe alone.
- Used for training very large AI models.

6. COMPARISON

Feature	PCIe	NVLink	NVSwitch
Purpose	GPU to CPU	GPU to GPU	Connects many GPUs
Typical Bandwidth (per direction)	Gen4: ~16 GB/s Gen5: ~32 GB/s	~50-900 GB/s (depends on GPU)	Up to several TB/s total
Scope	Single GPU	Direct GPU link	Multi-GPU switch
Common Use	Most servers	High-performance AI servers	Large AI clusters
Benefit	Standard and widely available	Faster GPU-to-GPU data transfer	Scales to many GPUs with high bandwidth

7. WHY HARDWARE CONNECTIVITY MATTERS

- Faster connections reduce data transfer time.
- High bandwidth is critical for large AI models and distributed training.
- Poor connectivity can create performance bottlenecks.
- Multi-GPU workloads rely on NVLink or NVSwitch for efficient scaling.
- The right hardware design improves training and inference performance.

8. KEY TAKEAWAYS

- ✓ PCIe connects GPUs to the CPU and is used in most servers.
- ✓ NVLink connects GPUs directly for high-speed communication.
- ✓ NVSwitch connects many GPUs together in large AI servers.
- ✓ Better hardware connectivity = better performance for AI workloads.
- ✓ Choose the right server design based on your workload size and scale.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

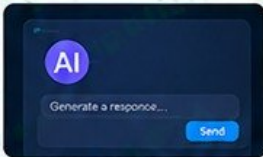
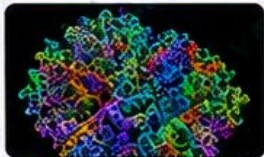


@veriqta

06 Types of GPU Workloads

AI training, inference, rendering, scientific computing and workload requirements.

1. COMMON GPU WORKLOAD TYPES

AI TRAINING	AI INFERENCE	RENDERING & MEDIA	SCIENTIFIC COMPUTING	OTHER WORKLOADS
				
- Used to train large AI models with massive datasets. - Requires high compute, high VRAM and long runtimes. - Examples: LLMs, computer vision, speech models.	- Used to run trained models and make predictions. - Requires lower compute than training but still benefits from GPUs. - Examples: chatbots, image generation, recommendation systems.	- Used for 3D rendering, video editing and visual effects. - Requires high compute and high memory bandwidth. - Examples: animation, movie production, architectural visualization.	- Used for simulations and complex scientific calculations. - Requires high compute and large memory. - Examples: drug discovery, climate modeling, physics simulations.	- Data analytics and big data processing. - High-performance computing (HPC). - Financial modeling and simulations. - Genomics and bioinformatics.

2. WORKLOAD COMPARISON

Workload Type	Compute Requirement	VRAM Requirement	Typical Runtime	Example Models / Tools
AI Training	Very High	Very High (24 GB – 80 GB+)	Hours to Weeks	LLMs, Stable Diffusion
AI Inference	Medium – High	Medium – High (8 GB – 40 GB)	Milliseconds to Seconds	Llama, GPT, Vision Models
Rendering & Media	High	High (8 GB – 48 GB)	Minutes to Hours	Blender, Unreal Engine
Scientific Computing	High	High (16 GB – 80 GB+)	Hours to Days	Molecular Dynamics, Climate Models
Data Analytics / HPC	High	High (16 GB – 80 GB+)	Minutes to Days	RAPIDS, CUDA, MATLAB

3. REAL WORLD EXAMPLES

	Training an LLM Uses multiple GPUs for weeks to process huge text datasets.
	Running a Chatbot Uses GPUs for fast real-time inference.
	Rendering a 3D Scene Uses GPUs to render complex graphics quickly.
	Drug Discovery Uses GPUs to simulate molecular interactions.
	Data Processing Uses GPUs to process large datasets much faster.

4. KEY FACTORS THAT AFFECT GPU WORKLOADS

	Model Size - Larger models require more VRAM and compute. - Example: 7B vs 70B parameter models.		Batch Size - Larger batch sizes use more memory. - Affects throughput and performance.
	Precision - FP32 (high accuracy, more memory) - FP16 / BF16 (used in AI) - INT8 (lower memory, faster inference)		Dataset Size - Larger datasets require longer training time. - High-speed storage and networking are important.

5. GPU WORKLOAD REQUIREMENTS

	Compute Power - Many parallel cores - High FLOPS (FP16/FP32)		VRAM Capacity 8 GB – 80 GB+ - Higher VRAM for larger models and data
	Memory Bandwidth - Faster bandwidth improves performance - Important for training and large datasets		Cooling and Power - GPUs generate a lot of heat - Requires proper cooling and sufficient power (e.g. 300W – 700W per GPU)

6. KEY TAKEAWAYS

- ✓ Different workloads have different GPU requirements.
- AI training needs the highest compute and memory.
- Inference, rendering and scientific computing also benefit greatly from GPUs.
- Model size, batch size, precision and dataset size affect GPU resource needs.
- Choose the right GPU type based on your workload, performance and budget.

7. JUNIOR ENGINEER TIP

- 💡 Always understand your workload requirements before choosing a GPU.
- More GPU power is not always needed.
- Match the GPU to the task, model size and budget.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

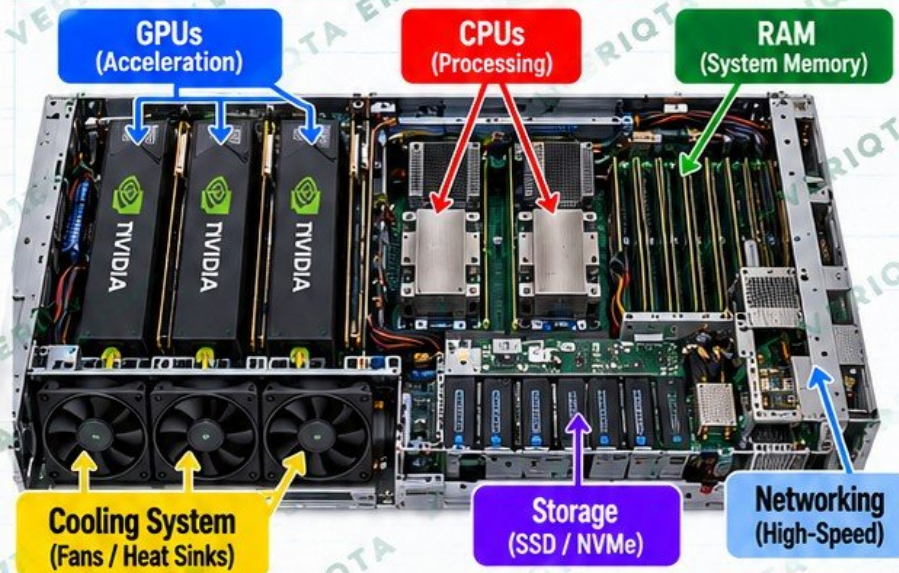
07 GPU Servers Explained

How GPUs, CPUs, RAM, storage, networking and cooling work together in one server.

1. GPU SERVER OVERVIEW

- A GPU server is a high-performance server designed to run workloads that require GPUs.
- It combines GPUs, CPUs, memory, storage, high-speed networking and advanced cooling.
- All components work together to process large amounts of data quickly and efficiently.
- Used for AI/ML, high-performance computing, rendering and more.

2. INSIDE A GPU SERVER (EXAMPLE)



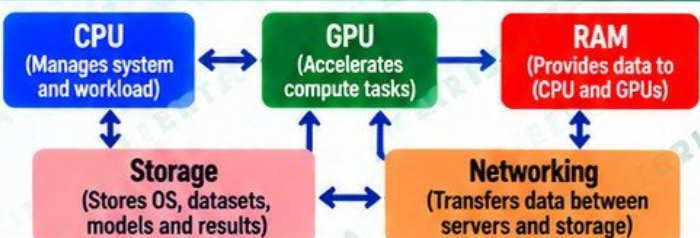
3. KEY COMPONENTS AND THEIR ROLES

GPUs	CPUs	RAM	Storage	Networking	Cooling System
					
● Accelerate parallel computations. ● Handle AI training, inference, rendering and simulations. ● Have their own high-speed memory (VRAM). ● Common models: NVIDIA A100, H100, L40, RTX 6000, etc.	● Handle general computing tasks. ● Manage the system and coordinate with GPUs. ● Provide high core count for data preparation and multi-threaded tasks. ● Common: Intel Xeon or AMD EPYC.	● System memory used by CPU and applications. ● Stores datasets, preprocessed data and application memory. ● Large capacity (e.g., 256 GB – 2 TB) is common in GPU servers.	● Stores operating system, datasets, models and results. ● High-speed NVMe SSDs are preferred. ● Large datasets require high storage capacity and fast read/write speeds.	● Provides high-speed data transfer. ● Common options: 10/25/40/100/200 GbE or InfiniBand. ● Important for distributed training and large datasets.	● GPUs generate a lot of heat. ● High-performance fans and heat sinks keep the server cool. ● Some servers use liquid cooling for very high-power GPUs.

4. TYPICAL GPU SERVER SPECIFICATIONS

Component	Example Specification
GPUs	4 × NVIDIA H100 (80 GB VRAM each)
CPUs	2 × Intel Xeon (64 cores each)
System RAM	512 GB – 2 TB DDR5
Storage	2 × 3.84 TB NVMe SSD (OS) 4 × 7.68 TB NVMe SSD (Datasets)
Networking	2 × 100 GbE or InfiniBand
Cooling	High-performance fans or liquid cooling
Power Supply	High wattage (e.g., 2000W – 3000W)
Form Factor	Rack-mounted (2U, 4U, etc.)

5. HOW THE COMPONENTS WORK TOGETHER



6. KEY TAKEAWAYS

- ✓ A GPU server is a combination of GPUs, CPUs, RAM, storage, networking and cooling.
- Each component has a specific role and all work together.
- The right configuration depends on the workload (training, inference, rendering, scientific computing, etc.).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

08

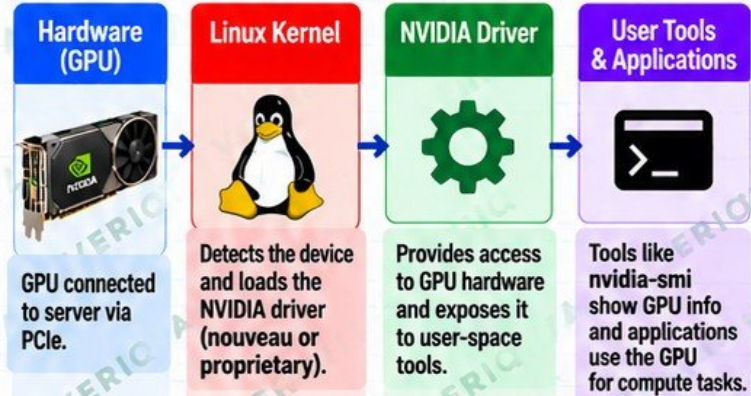
GPU Infrastructure on Linux

How Linux detects GPU hardware and why drivers and kernel compatibility matter.

1. HOW LINUX DETECTS GPU HARDWARE

- When you boot a Linux server, the kernel detects hardware devices on the PCIe bus.
- GPUs appear as PCI devices and are identified by their vendor and device IDs (e.g., NVIDIA).
- The kernel loads the appropriate drivers to enable communication with the GPU.
- User-space tools (like `nvidia-smi`) then interact with the driver to show GPU information.

2. GPU DETECTION FLOW IN LINUX



3. CHECK IF LINUX DETECTS YOUR GPU

```
$ lspci | grep -i nvidia
01:00.0 3D controller: NVIDIA Corporation
          RTX 6000 Ada Generation (rev a1)
```

```
$ lsmod | grep nvidia
```

```
nvidia           12345678  0
nvidia_uvm       2345678  0
nvidia_drm       345678  0
nvidia_modeset   456789  0
```

- `lspci` shows the GPU hardware on the PCIe bus.
- `lsmod` shows if the NVIDIA kernel modules are loaded.
- If you see `nvidia` in the output, Linux has detected the GPU and the driver is loaded.

4. USE NVIDIA-SMI TO VIEW GPU DETAILS

```
$ nvidia-smi
```

NVIDIA-SMI 550.54.14		Driver Version: 550.54.14		CUDA Version: 12.4	
GPU	Name	Persistence-M	Bus-Id	Memory-Usage	
0	NVIDIA RTX 6000	Off	00000000:01:00.0	120MiB / 49140MiB	
1	NVIDIA RTX 6000	Off	00000000:81:00.0	80MiB / 49140MiB	

- `nvidia-smi` shows GPU model, driver version, CUDA version, memory usage, and running processes.
- If `nvidia-smi` is not found, the driver is not installed or not loaded.

5. KEY LINUX FILES AND LOCATIONS

Path	Purpose
<code>/dev/nvidia*</code>	GPU device files (used by applications)
<code>/proc/driver/nvidia/</code>	Driver and GPU information
<code>/sys/bus/pci/devices/</code>	PCI device information
<code>/var/log/nvidia-installer.log</code>	Driver installation log
<code>/etc/modprobe.d/</code>	NVIDIA kernel module configuration
<code>/usr/bin/nvidia-smi</code>	GPU management tool
<code>/usr/local/cuda/</code>	CUDA toolkit (if installed)

6. DRIVER AND KERNEL COMPATIBILITY

Component	Role	Why It Matters
Linux Kernel	Manages hardware and loads drivers.	Driver must support your kernel version.
NVIDIA Driver	Provides access to the GPU.	Wrong driver version can cause failures or no GPU detection.
CUDA Toolkit	User-space tools and libraries.	Must be compatible with the driver version.
Applications	Use CUDA, cuDNN, NCCL, etc.	Must match driver and CUDA versions.

7. COMMON ISSUES AND SOLUTIONS

Issue	Solution
GPU not detected	Check hardware, PCIe, BIOS, and driver installation.
<code>nvidia-smi</code> not found	Driver not installed or not loaded. Install NVIDIA driver.
Driver fails to load	Kernel version not supported. Use a compatible driver or update the kernel.
CUDA version mismatch	Use a CUDA version that matches your driver.
GPU shows but not usable	Check permissions, <code>/dev/nvidia*</code> and user groups.
System boots with nouveau	Disable nouveau and install official NVIDIA driver.

8. KEY TAKEAWAYS

- Linux detects GPUs through the PCIe bus.
- NVIDIA drivers are required to use the GPU.
- Driver and kernel version compatibility is critical.
- `nvidia-smi` is the main tool to verify GPU availability.
- Incorrect drivers or kernel versions can cause detection and performance issues.
- Always use the correct driver and CUDA version for your Linux distribution and kernel.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

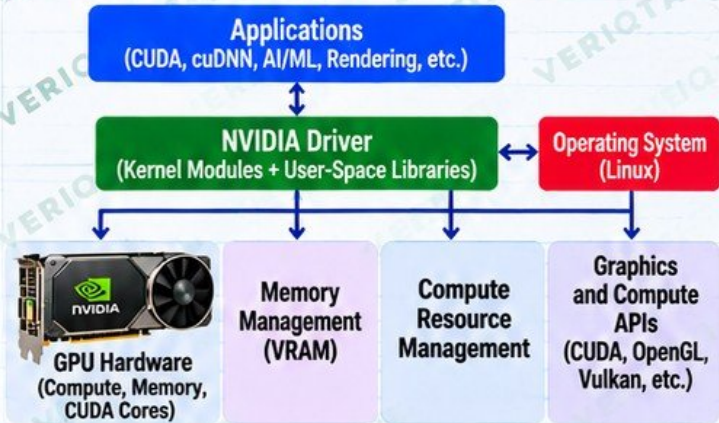
09 NVIDIA Drivers Explained

What GPU drivers do, installation concepts, compatibility and common driver failures.

1. WHAT ARE NVIDIA DRIVERS?

- NVIDIA drivers are software that allow the operating system and applications to communicate with the GPU.
- They manage GPU hardware, memory, compute resources and features like CUDA, cuDNN and NVIDIA libraries.
- Without the correct driver, the GPU will not be detected or will not work properly.
- Drivers provide stability, performance, bug fixes and support for new GPU features.

2. HOW NVIDIA DRIVERS WORK



3. KEY DRIVER COMPONENTS



Kernel Modules

- Loaded into the Linux kernel.
- Handle low-level GPU hardware communication.
- Examples: nvidia, nvidia_uvm, nvidia_drm, nvidia_modeset.



User-Space Libraries

- Used by applications and tools.
- Include CUDA, cuDNN, NCCL, NVML, OpenCL, etc.
- Provide APIs for GPU compute and management.



Management Tools

- nvidia-smi (check GPU status)
- nvidia-settings (configure GPU)
- nvidia-persistenced (keep driver loaded)
- NVIDIA Container Toolkit (for containers)

4. DRIVER VERSIONS AND COMPATIBILITY

Driver Type	Purpose	Best For
Production Branch (PB)	Stable and tested releases.	Production servers.
New Feature Branch (NFB)	Latest features and improvements.	New hardware and latest features.
Long Lived Branch (LLB)	Extended support and stability.	Enterprise and long-term deployments.

COMPATIBILITY FACTORS

- ✓ Driver version must match your GPU model.
- ✓ Driver must support your Linux kernel version.
- ✓ CUDA version must be compatible with the driver.
- ✓ Some applications (e.g., PyTorch, TensorFlow) require specific CUDA and driver versions.

5. CHECK INSTALLED DRIVER VERSION

```

$ nvidia-smi
+-----+-----+
| NVIDIA-SMI 550.54.14 | Driver Version: 550.54.14 |
+-----+-----+
| GPU   Name           | Persistence-M | Memory-Usage |
+-----+-----+
| 0     NVIDIA RTX 6000 | On            | 0MiB / 49140MiB |
+-----+-----+
$ nvidia-smi --query-gpu=name,driver_version --format=csv
  
```

6. INSTALLATION CONCEPTS (UBUNTU EXAMPLE)

```

# Update system
sudo apt update
sudo apt install -y build-essential dkms

# Install NVIDIA driver (example)
sudo ubuntu-drivers devices
sudo apt install -y nvidia-driver-550

# Reboot and verify
sudo reboot
nvidia-smi
  
```

- Use the recommended driver for your GPU.
- DKMS helps rebuild the driver when the kernel is updated.
- Always reboot after installation.
- Verify with nvidia-smi.

7. COMMON DRIVER FAILURES

Issue	Possible Causes	Solution
GPU not detected	Driver not installed, wrong driver version, GPU not supported	Install correct driver, check hardware and PCIe, verify with lspci
nvidia-smi not found	Driver not installed or not in PATH	Install driver and check environment
Driver fails to load	Kernel version mismatch	Use compatible driver or update kernel
CUDA not working	CUDA version mismatch	Install matching CUDA and driver versions
GPU shows but not usable	Permission issues, driver error	Check user groups, reload driver, check logs
System boots with errors	Nouveau driver conflict	Disable nouveau and install NVIDIA driver

8. KEY TAKEAWAYS

- ✓ NVIDIA drivers enable communication between the OS, applications and the GPU.
- ✓ Driver and kernel compatibility is critical.
- ✓ Use the correct driver version for your GPU model and Linux distribution.
- ✓ Always verify installation with nvidia-smi.
- ✓ Most driver issues are caused by version mismatches, missing dependencies or conflicting drivers.
- ✓ Keep drivers updated for stability, performance and security fixes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

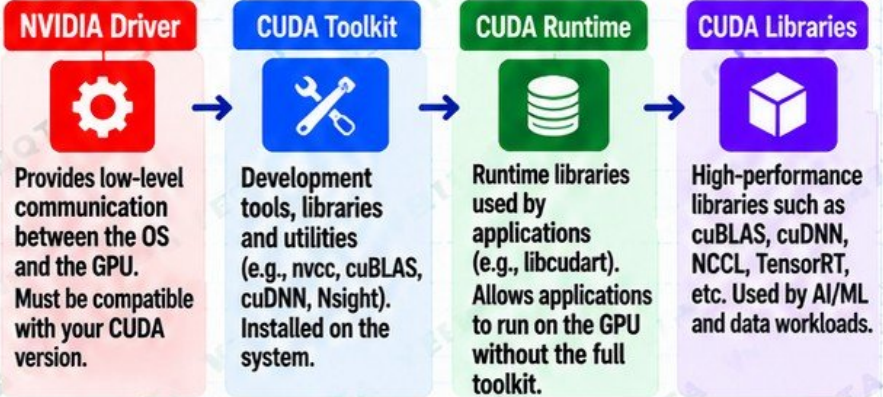
10 CUDA Infrastructure Basics

CUDA, the CUDA Toolkit, drivers and runtime compatibility, without teaching GPU programming.

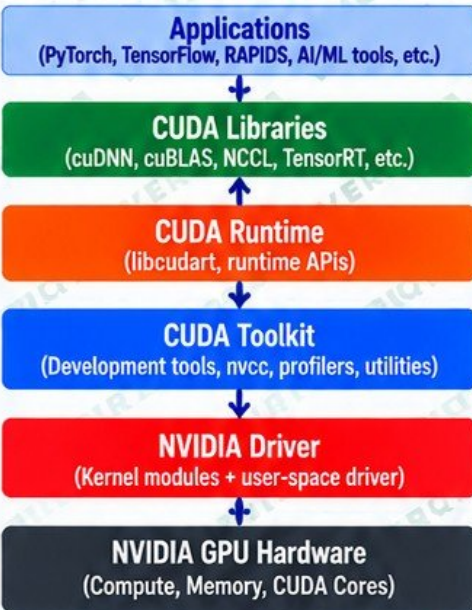
1. WHAT IS CUDA?

- CUDA (Compute Unified Device Architecture) is NVIDIA's platform for GPU computing.
- It provides libraries, tools and a runtime environment that allow applications to use NVIDIA GPUs.
- CUDA is used by AI/ML frameworks, scientific computing tools, data analytics platforms and more.
- You do not need to write CUDA code to use most AI tools, but you need the right CUDA infrastructure installed and configured.

2. CUDA INFRASTRUCTURE COMPONENTS



3. HOW CUDA FITS INTO THE STACK



4. DRIVER, TOOLKIT AND RUNTIME RELATIONSHIP



5. VERSION COMPATIBILITY (OVERVIEW)

CUDA Version	Minimum NVIDIA Driver (Linux x86_64)	Example Toolkit Version	Common Use Cases
CUDA 11.8	515.43 or later	11.8	Stable for many AI/ML tools
CUDA 12.0	525.60 or later	12.0	Newer frameworks and features
CUDA 12.1	530.30 or later	12.1	Improved performance and tools
CUDA 12.2	535.54 or later	12.2	Latest libraries and optimizations
CUDA 12.3	535.104 or later	12.3	Recent GPUs and AI workloads

- The NVIDIA driver must be compatible with the CUDA version.
- Newer CUDA toolkits may require newer driver versions.
- You can usually run applications with an older toolkit if the driver supports it, but check NVIDIA's compatibility matrix.

6. VERIFY CUDA ON YOUR LINUX SERVER

```

# Check NVIDIA driver and GPU
nvidia-smi
+-----+-----+-----+
| NVIDIA-SMI 550.54.14 | Driver Version: 550.54.14 | CUDA Version: 12.4 |
+-----+-----+-----+

# Check CUDA compiler (if toolkit installed)
nvcc --version
nvcc: NVIDIA (R) Cuda compilation driver
Cuda compilation tools, release 12.4, V12.4.131

# Check CUDA runtime library
ldconfig -p | grep libcudart
libcudart.so.12 (libc6,x86_64) => /usr/local/cuda/lib64/libcudart.so.12
libcudart.so (libc6,x86_64) => /usr/local/cuda/lib64/libcudart.so
  
```

8. KEY TAKEAWAYS

- ✓ CUDA provides the platform, libraries and tools to use NVIDIA GPUs.
- ✓ The NVIDIA driver must be compatible with your CUDA version.
- ✓ The CUDA toolkit includes development tools and runtime libraries.
- ✓ Most AI/ML tools use CUDA libraries and runtime, not the full toolkit.
- ✓ Always verify driver, toolkit and runtime versions on your Linux server.
- ✓ Use NVIDIA's official compatibility documentation for supported versions.

7. COMMON CUDA RELATED ISSUES

Issue	Possible Cause and Solution
nvidia-smi not found	NVIDIA driver not installed or not in PATH. Install driver and verify.
CUDA driver version is insufficient	Your driver is older than required for the CUDA toolkit. Update the NVIDIA driver.
nvcc not found	CUDA toolkit is not installed or not in PATH. Install toolkit or add to PATH.
Application cannot find libcudart.so	CUDA runtime libraries not installed or wrong version. Install the correct CUDA toolkit or runtime package.
CUDA version mismatch	Application requires a specific CUDA version. Install a compatible driver and toolkit version.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

11 Your First GPU Commands

Using nvidia-smi to inspect GPU availability, utilization, memory and processes.

1. WHAT IS nvidia-smi?



- nvidia-smi (NVIDIA System Management Interface) is a command-line tool provided by NVIDIA.

- It allows you to check GPU hardware, driver information, utilization, memory usage, processes and more.
- It is the first tool you should use to verify that your GPU is working correctly on a Linux server.
- It does not require you to write GPU code.

2. CHECK GPU AVAILABILITY

\$ nvidia-smi

NVIDIA-SMI 550.54.14			Driver Version: 550.54.14			CUDA Version: 12.4		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC	GPU-Util	Compute M.
Fan	Temp		Pwr:Usage/Cap		Memory-Usage			MIG M.
0	NVIDIA RTX 6000	Off	00000000:81:00:0	On		N/A		
30%	45C	P8	28W / 300W	1250MiB / 49140MiB	12%	Default		N/A

- Shows GPU model, driver version and CUDA version.
- Displays real-time utilization, memory usage, temperature and power.

3. CHECK DETAILED GPU INFORMATION

\$ nvidia-smi -L

GPU 0: NVIDIA RTX 6000 (UUID: GPU-ad3b2f9e-1c4a-11ee-8c99-0242ac120002)
GPU 1: NVIDIA RTX 6000 (UUID: GPU-9f7c6e2d-3d1b-11ee-8a44-0242ac120003)



- Lists all available GPUs with their model names and UUIDs.
- Useful when you have multiple GPUs in the server.

4. CHECK GPU UTILIZATION (LIVE VIEW)

\$ nvidia-smi -l 1

NVIDIA-SMI 550.54.14			Driver Version: 550.54.14			CUDA Version: 12.4		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC	GPU-Util	Compute M.
Fan	Temp		Pwr:Usage/Cap		Memory-Usage			MIG M.
0	NVIDIA RTX 6000	Off	00000000:81:00:0	On		N/A		
38%	47C	P2	120W / 300W	20480MiB / 49140MiB	78%	Default		N/A



- Updates GPU statistics every second.
- Helps you monitor real-time usage, temperature and memory.

5. CHECK GPU MEMORY USAGE

\$ nvidia-smi --query-gpu=memory.used,memory.total \\
--format=csv
memory.used [MiB], memory.total [MiB]
1250, 49140
20480, 49140



- Shows GPU memory used and total memory.
- Useful for quick checks and scripting.

6. CHECK RUNNING PROCESSES ON THE GPU

\$ nvidia-smi

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
	ID	ID				
0	N/A	N/A	2345	C	python	10240MiB
0	N/A	N/A	5678	C	/usr/bin/ollama	5120MiB



- Shows which processes are using the GPU.
- Useful to identify applications consuming GPU memory.

7. CHECK DRIVER AND CUDA VERSION

\$ nvidia-smi
Driver Version: 550.54.14
CUDA Version: 12.4



- Driver version must be compatible with your CUDA toolkit.
- Different CUDA versions require minimum driver versions.

9. KEY TAKEAWAYS

- ✓ nvidia-smi is the main tool to inspect GPU availability and usage.
- ✓ It shows GPU model, driver version, CUDA version, memory, utilization and running processes.
- ✓ Use it regularly to monitor GPU health and performance.
- ✓ Driver and CUDA version compatibility is critical.
- ✓ It works on all NVIDIA GPUs and does not require programming.

8. OTHER USEFUL nvidia-smi COMMANDS

Command	Purpose
nvidia-smi -L	List all GPUs.
nvidia-smi -q	Show detailed information about GPUs.
nvidia-smi --query-gpu=name,memory.total --format=csv	Show GPU names and total memory.
nvidia-smi topo -m	Show GPU topology (useful for multi-GPU servers).
nvidia-smi -pm 1	Enable persistence mode.
nvidia-smi --gpu-reset -i 0	Reset a specific GPU (use with caution).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

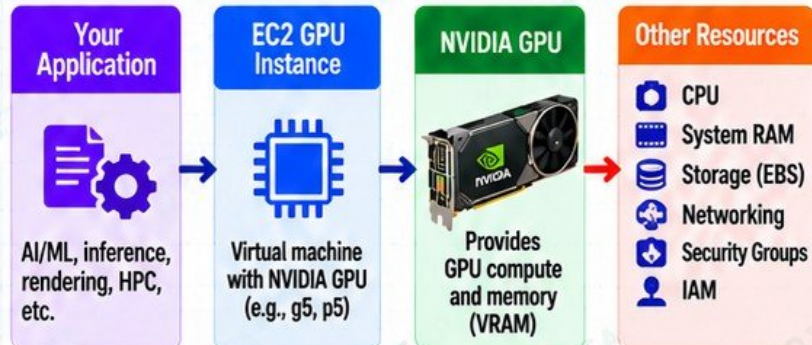
12 GPU Infrastructure in the Cloud

GPU-enabled virtual machines, AWS EC2 GPU instance families and basic provisioning decisions.

1. WHAT ARE GPU INSTANCES?

- GPU instances are virtual machines with dedicated NVIDIA GPUs.
- They provide the same GPU capabilities as on-premises servers, but in the cloud.
- Used for AI/ML, inference, rendering, scientific computing and more.
- You choose the instance type based on your workload, GPU type, memory and cost.

2. HOW A CLOUD GPU INSTANCE WORKS (AWS EC2)



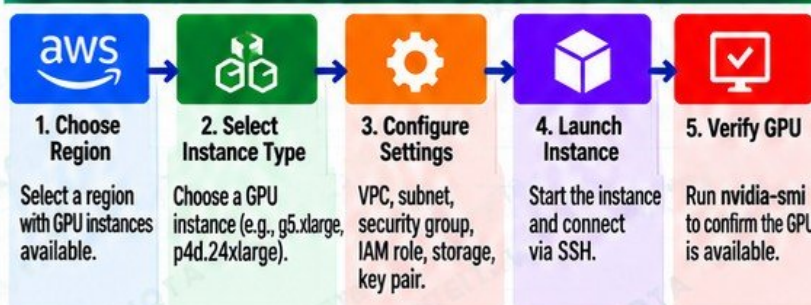
3. COMMON AWS EC2 GPU INSTANCE FAMILIES

Instance Family	GPU Type	Use Cases	Key Features
P5	NVIDIA H100	Large AI training, foundation models	Highest performance, NVLink, large VRAM (80 GB)
P4d	NVIDIA A100	AI training, HPC	Multi-GPU, high bandwidth (EFA networking)
G5	NVIDIA A10G	Inference, mid-size training, graphics	Cost-effective, versatile
G6 (coming/next)	NVIDIA L40S	AI inference, media, graphics, generative AI	Latest generation GPU
G4	NVIDIA T4	Inference, light training, video processing	Lower cost, good for small workloads

4. INSTANCE SELECTION FACTORS

- WORKLOAD TYPE**
Training, inference, rendering, HPC, etc.
- GPU TYPE AND VRAM**
Choose the right GPU and memory size for your workload.
- CPU, RAM AND STORAGE**
Ensure enough vCPUs, RAM and storage for your application.
- NETWORKING REQUIREMENTS**
Use EFA for distributed training when needed.
- COST AND AVAILABILITY**
Compare on-demand, reserved and spot instances.

5. LAUNCHING A GPU INSTANCE (AWS CONSOLE)



6. VERIFY GPU ON YOUR INSTANCE

```
$ nvidia-smi
```

NVIDIA-SMI 550.54.14		Driver Version: 550.54.14		CUDA Version: 12.4
GPU	Name	Persistence-M	Bus-Id	Memory-Usage
0	NVIDIA A10G	Off	00000000:00:18:0	0MiB / 23028MiB

- Confirms the GPU is detected and working.
- Shows GPU model, driver version, CUDA version and memory.
- Run this after launching the instance to verify your setup.

7. COMMON USE CASES AND EXAMPLES

Use Case	Example Instance	Description
AI Model Training	p5.48xlarge	NVIDIA H100, high performance for large models.
AI Model Inference	g5.xlarge	NVIDIA A10G, cost-effective for inference.
Machine Learning Development	g5.2xlarge	Good balance of cost and performance.
Rendering / Graphics	g5.xlarge or g6	Graphics, media processing, 3D rendering.
Scientific Computing	p4d.24xlarge	NVIDIA A100, high-performance computing.

8. IMPORTANT CONSIDERATIONS

- Not all regions have GPU instances. Check availability.
- Use the correct AMI with NVIDIA drivers or install drivers after launch.
- Choose the right instance size based on your workload.
- Consider EFA for distributed training workloads.
- Manage costs using reserved or spot instances.
- Monitor GPU utilization and costs.

9. KEY TAKEAWAYS

- GPU instances provide NVIDIA GPUs in the cloud.
- AWS offers multiple GPU instance families for different workloads.
- Choose instance type based on workload, performance and cost.
- Verify GPU availability with nvidia-smi.
- Consider region availability, networking and cost when provisioning.

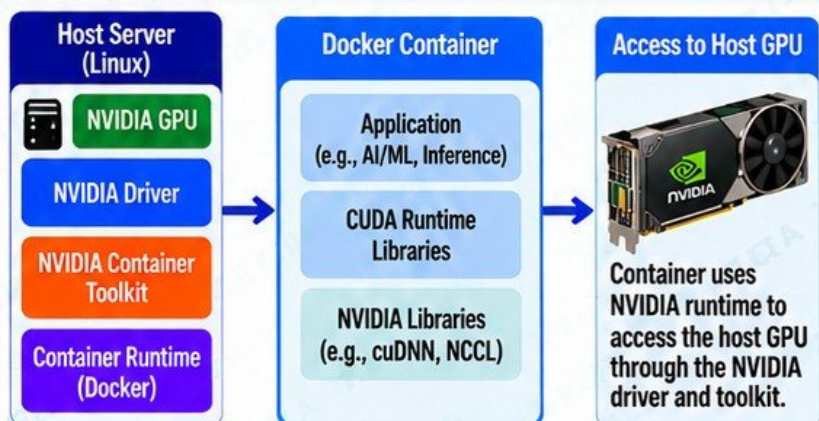
13 GPUs and Docker

GPU-enabled containers, NVIDIA Container Toolkit and how applications access host GPUs.

1. WHY USE GPUS WITH DOCKER?

- Run GPU workloads in containers.
- Consistent and reproducible environments.
- Easier dependency management.
- Run multiple applications on the same GPU server.
- Isolate applications for security and stability.

2. HOW GPU CONTAINERS WORK



3. NVIDIA CONTAINER TOOLKIT COMPONENTS

- nvidia-container-toolkit**
Enables GPU support in Docker containers.
- nvidia-container-runtime**
Configures the container runtime to use GPUs.
- libnvidia-container**
Handles GPU device discovery and mounting.
- nvidia-docker (legacy)**
Older tool, now replaced by the NVIDIA Container Toolkit.

4. INSTALL NVIDIA CONTAINER TOOLKIT (UBUNTU)

```

# Add NVIDIA package repository
curl -fsSL https://nvidia.github.io/nvidia-docker/gpgkey | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit.gpg

# Add repository
curl -s -L https://nvidia.github.io/nvidia-docker/ubuntu20.04/nvidia-docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list

# Install toolkit
sudo apt update
sudo apt install -y nvidia-container-toolkit

# Configure Docker
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
  
```

- Installs the NVIDIA Container Toolkit.
- Configures Docker to use NVIDIA runtime.
- Allows containers to access the host GPU.
- Works with recent NVIDIA drivers and CUDA versions.

5. RUN A GPU-ENABLED CONTAINER

```

# Run an NVIDIA CUDA container
docker run --rm --gpus all \
  nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
  
```

- gpus all gives the container access to all GPUs.
- The container uses the host NVIDIA driver.
- The image provides CUDA runtime libraries.

6. VERIFY GPU ACCESS INSIDE THE CONTAINER

```

# Check GPU inside the container
docker run --rm --gpus all \
  nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
  
```

NVIDIA-SMI 550.54.14 Driver Version: 550.54.14 CUDA Version: 12.4				
GPU Name	Persistence-M	Bus-Id	Disp.A	Memory-Usage
0 NVIDIA RTX 6000	On	00000000:81:00.0	Off	1024MiB / 49140MiB

7. COMMON USE CASES

- AI/ML Training**
Run training frameworks (PyTorch, TensorFlow) with GPU support.
- Inference Services**
Deploy inference models with GPU acceleration.
- Scientific Computing**
Run simulations, CUDA applications and HPC workloads.
- Media Processing**
Video transcoding, rendering and graphics workloads.
- Multiple Workloads**
Run multiple isolated applications on the same GPU server.

8. IMPORTANT CONSIDERATIONS

- Driver Compatibility**
Host driver must be compatible with the container CUDA version.
- Container Images**
Use official NVIDIA images or trusted images with CUDA libraries.
- GPU Access Control**
Limit GPU access using Docker security options when needed.
- Resource Management**
Monitor GPU usage to prevent resource contention.
- Version Matching**
Match driver, CUDA and container versions for stability.

9. KEY TAKEAWAYS

- Docker can run GPU-enabled containers using the NVIDIA Container Toolkit.
- Containers access the host GPU through the NVIDIA driver.
- Use --gpus flag to run GPU containers.
- Driver and CUDA version compatibility is critical.
- GPU containers are ideal for AI/ML, inference, scientific computing and media workloads.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

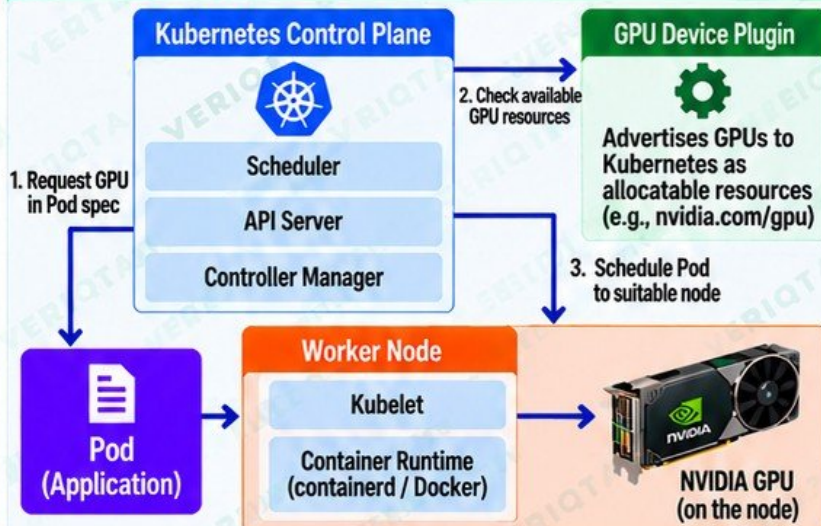
14 GPUs in Kubernetes

GPU device plugins, GPU resource requests and how Kubernetes schedules GPU workloads.

1. WHY USE GPUS IN KUBERNETES?

- Run AI/ML, inference, rendering and HPC workloads at scale.
- Efficient resource utilization.
- Automatic scheduling and self-healing.
- Isolation between teams and workloads.
- Manage GPU workloads using the same Kubernetes tools.

2. GPU WORKFLOW IN KUBERNETES



3. NVIDIA GPU DEVICE PLUGIN

- Runs as a DaemonSet on each worker node.
- Detects available GPUs on the node.
- Advertises GPUs to Kubernetes as nvidia.com/gpu resources.
- Works with NVIDIA Container Toolkit and the host driver.

4. GPU RESOURCE MODEL

Resource Name	Description	Example
nvidia.com/gpu	Number of GPUs to allocate	1, 2, 4
limits	Maximum GPUs for the container	nvidia.com/gpu: 1
requests	Minimum GPUs required	nvidia.com/gpu: 1

5. HOW SCHEDULING WORKS

- You request GPUs in the Pod specification.
- Kubernetes scheduler checks nodes with available GPUs.
- Pod is scheduled to a node with enough free GPUs.
- Device plugin makes the GPUs available to the container.
- Container runtime uses the NVIDIA driver and runtime to provide GPU access to the pod.

6. EXAMPLE POD SPEC WITH GPU REQUEST

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  containers:
    - name: app
      image: nvidia/cuda:12.4.1-base-ubuntu22.04
      command: ["nvidia-smi"]
      resources:
        limits:
          nvidia.com/gpu: 1
        requests:
          nvidia.com/gpu: 1
```



- Requests 1 GPU for the container.
- Uses an NVIDIA CUDA image.
- Runs nvidia-smi to verify GPU access.

7. VERIFY GPU IN A KUBERNETES POD

```
# Create the pod
kubectl apply -f gpu-pod.yaml

# Check pod status
kubectl get pods

# View GPU inside the pod
kubectl exec -it gpu-pod -- nvidia-smi
```



- Confirms the pod can access the GPU.
- Shows driver version, CUDA version and GPU details.
- Use this to verify correct setup.

NVIDIA-SMI 550.54.14		Driver Version: 550.54.14		CUDA Version: 12.4	
GPU Name	Persistence-M	Bus-Id	Disp.A	Memory-Usage	
0 NVIDIA RTX 6000	On	00000000:81:00.0	Off	1024MiB / 4234MiB	

8. IMPORTANT CONSIDERATIONS

- Use GPU-enabled nodes (with NVIDIA drivers and device plugin).
- Ensure compatible CUDA, driver and container runtime versions.
- Use node selectors, labels or taints to control where GPU pods run.
- Plan for resource limits, multi-tenancy and fair allocation.
- Monitor GPU usage and avoid resource contention.

9. COMMON USE CASES

- AI/ML Training**
Large model training with distributed GPUs.
- Inference Services**
Deploy models for real-time inference.
- Scientific Computing**
Simulations, research and data analysis.
- Rendering / Graphics**
3D rendering and media workloads.
- Multi-Tenant Clusters**
Run GPU workloads for multiple teams.

10. KEY TAKEAWAYS

- GPU device plugins make GPUs available to Kubernetes.
- You request GPUs using nvidia.com/gpu.
- Kubernetes schedules pods to nodes with available GPUs.
- Use compatible drivers, CUDA and container runtime.
- Monitor, manage and secure GPU workloads in production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

15 Sharing GPU Resources

Dedicated GPUs, NVIDIA MIG, time-slicing, isolation and resource-sharing trade-offs.

1. WHY SHARE GPU RESOURCES?



- Improve GPU utilization and reduce idle resources.



- Lower infrastructure costs.



- Support multiple workloads on the same GPU server.



- Give teams the right level of isolation and performance.



- Choose the best sharing method based on your workload requirements.

2. GPU SHARING METHODS OVERVIEW

Dedicated GPU

One workload per GPU.



- Full GPU resources
- Best performance
- Strong isolation
- Higher cost
- Ideal for large or sensitive workloads

NVIDIA MIG

Split one GPU into multiple isolated GPU instances.



- Multiple isolated GPU instances
- Hardware-level isolation
- Predictable performance
- Supported on NVIDIA A100, H100
- Ideal for multi-tenant environments

Time-Slicing

Share one GPU among multiple workloads over time.



- Multiple workloads share the same GPU
- No hardware isolation
- Good for non-critical and flexible workloads
- Higher utilization
- Performance can vary depending on workloads

3. DEDICATED GPU (ONE WORKLOAD PER GPU)



- Entire GPU is allocated to one workload.
- Maximum performance and stability.
- Strong isolation between workloads.
- Best for large training jobs or latency-sensitive inference.
- Higher cost per workload.

4. NVIDIA MIG (MULTI-INSTANCE GPU)



GPU with MIG Enabled

MIG 1
(1g.5gb)

MIG 2
(2g.10gb)

MIG 3
(3g.20gb)

MIG 4
(7g.40gb)

- MIG partitions one physical GPU into multiple isolated instances.
- Each instance has dedicated compute, memory and cache.
- Hardware-level isolation for security and stability.
- Ideal for multi-tenant environments (e.g., Kubernetes, cloud).
- Supported on NVIDIA A100 and H100 (and newer GPUs).

5. TIME-SLICING (MULTIPLE WORKLOADS SHARE A GPU)



- Multiple workloads share the same GPU over time.
- Improves GPU utilization.
- No hardware isolation between workloads.
- Suitable for development, testing and non-critical workloads.
- Performance can be affected by other workloads.

6. COMPARISON OF GPU SHARING METHODS

Feature	Dedicated GPU	NVIDIA MIG	Time-Slicing
Resource Allocation	Entire GPU	Partitioned GPU	Shared over time
Isolation	Strong	Hardware-level	None (software)
Performance	Highest	Predictable	Variable
GPU Utilization	Lower	Higher	Highest
Cost per Workload	Higher	Lower	Lower
Best Use Cases	Large training, critical inference	Multi-tenant, production	Development, testing, non-critical

7. WHEN TO USE EACH METHOD

Use Dedicated GPU

- ✓ Large AI training jobs
- ✓ Latency-sensitive inference
- ✓ Workloads requiring full GPU resources
- ✓ Strong isolation required

Use NVIDIA MIG

- ✓ Multiple teams or tenants
- ✓ Need hardware isolation
- ✓ Predictable performance
- ✓ Better GPU utilization
- ✓ Supported GPUs (A100, H100)

Use Time-Slicing

- ✓ Development and testing
- ✓ Non-critical workloads
- ✓ Cost optimization
- ✓ Flexible performance requirements
- ✓ High GPU utilization

8. KEY TAKEAWAYS

- ✓ You can share GPU resources using dedicated GPUs, NVIDIA MIG or time-slicing.
- ✓ MIG provides hardware-level isolation and predictable performance.
- ✓ Time-slicing increases utilization but has no hardware isolation.
- ✓ Choose the right method based on your workload requirements, isolation needs and cost.
- ✓ Proper GPU sharing allows you to support more workloads and get better value from your GPU infrastructure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

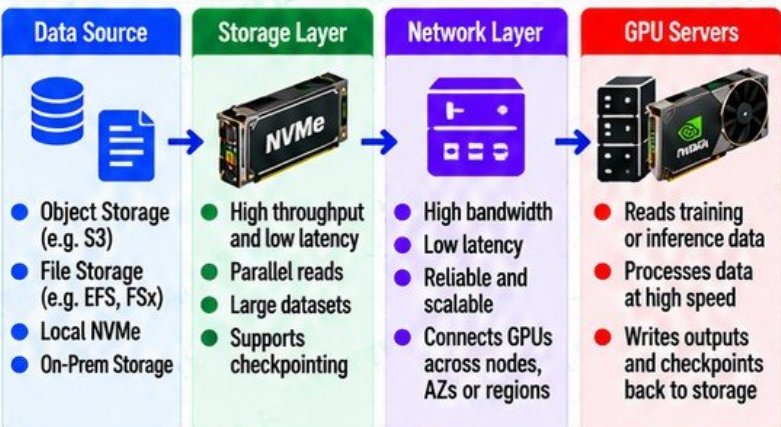
16 GPU Storage and Networking

Why datasets, storage throughput, network bandwidth and latency affect GPU workloads.

1. WHY STORAGE AND NETWORKING MATTER

- GPUs are very fast, but they depend on data. If data cannot be read, written or transferred fast enough, the GPU will sit idle.
- Storage throughput and network bandwidth directly affect training time, inference performance and overall cost.
- Low storage or network performance creates bottlenecks, increases job duration and reduces GPU utilization.
- Proper storage and networking architecture ensures GPUs are always fed with data and can work efficiently.

2. GPU WORKLOAD DATA FLOW



3. STORAGE OPTIONS FOR GPU WORKLOADS

Local NVMe SSD	Network File Storage	Object Storage	Block Storage
 ● Highest throughput ● Lowest latency ● Best for temporary datasets, training and scratch space	 ● AWS EFS / FSx ● Shared access across multiple instances ● Good for training datasets and collaboration	 ● AWS S3 ● Highly scalable ● Cost-effective ● Ideal for large datasets, model artifacts and backups	 ● EBS (gp3, io2) ● Persistent storage for GPU instances ● Good for model storage, logs and data pipelines

4. NETWORKING OPTIONS FOR GPU WORKLOADS

Single Node	Multi-Node (Same Region)	Multi-AZ / Multi-Region	On-Prem / Hybrid
 ● Local PCIe / NVLink ● Low latency ● Used for 1-8 GPUs on the same server	 ● 25 / 100 / 200 Gbps EFA / ENA ● Low latency, high throughput ● Used for distributed training and inference	 ● VPC, Direct Connect ● Secure and reliable ● Higher latency than same region ● Used for large scale and global workloads	 ● Dedicated links ● High bandwidth ● Used for hybrid cloud or on-prem GPU clusters

5. STORAGE THROUGHPUT EXAMPLE (READING DATA)

```
# Test read speed using dd
sudo dd if=/dev/nvme0n1 of=/dev/null bs=1G count=5

# Check disk performance
sudo fio --name=readtest --filename=/mnt/data/testfile \
--rw=read --bs=1M --size=10G --numjobs=4 --runtime=60 --group_reporting
```

Storage Type	Example	Typical Throughput	Typical Latency
Local NVMe	Instance NVMe SSD	3 – 7 GB/s	< 1 ms
EBS gp3	AWS EBS gp3	250 – 1,000 MB/s	1 – 2 ms
EBS io2	AWS EBS io2	500 – 4,000 MB/s	< 1 ms
EFS	AWS EFS	100 – 1,000 MB/s	1 – 10 ms
S3 (via network)	AWS S3	Varies (100 MB/s – GB/s)	10 – 100 ms

6. NETWORK BANDWIDTH AND LATENCY EXAMPLE

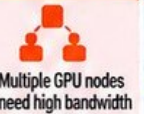
```
# Test network bandwidth (iperf3)
iperf3 -c <server-ip> -t 30 -P 4

# Test network latency (ping)
ping -c 10 <server-ip>

# Test latency with detailed stats
mtr <server-ip>
```

Network Type	Example	Bandwidth	Typical Latency
Local (same server)	PCIe / NVLink	Up to 900 GB/s	< 1 μs
Same region	EFA (AWS)	25 – 200 Gbps	< 1 ms
VPC (same AZ)	ENA	Up to 100 Gbps	< 1 ms
VPC (cross AZ)	AWS network	Up to 100 Gbps	1 – 2 ms
Direct Connect	On-prem to AWS	1 – 100 Gbps	2 – 10 ms
Internet	Public network	Varies	10 – 100+ ms

7. IMPACT ON GPU WORKLOADS

Training Large Models	Inference Services	Distributed Training	Data Preprocessing
 ● Requires high storage throughput and high network bandwidth. ● Slow storage or network increases training time.	 ● Needs low latency storage and network for fast response. ● High latency can cause slow inference and timeouts.	 ● Multiple GPU nodes need high bandwidth and low latency. ● Uses EFA, InfiniBand or high-speed networking.	 ● Reading and preparing datasets must be fast. ● Poor storage performance can create GPU idle time.

8. BEST PRACTICES

- ✓ Use fast storage (NVMe or high-performance EBS) for training and inference.
- ✓ Use high bandwidth, low latency networking (EFA, InfiniBand) for distributed workloads.
- ✓ Keep datasets close to the GPUs.
- ✓ Use parallel data loading and multiple workers.
- ✓ Monitor storage and network performance.
- ✓ Plan for scalability as dataset size grows.
- ✓ Choose the right storage and network based on your workload requirements.

9. KEY TAKEAWAYS

- ✓ GPUs need fast storage and networking to stay busy.
- ✓ Storage throughput and network bandwidth directly affect training and inference performance.
- ✓ Low storage or high network latency creates bottlenecks.
- ✓ Use the right storage and network solution for your workload and scale.
- ✓ Always monitor and optimize storage and network performance.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

17 Monitoring GPU Infrastructure

GPU utilization, memory usage, temperature, power, NVIDIA DCGM and Prometheus.

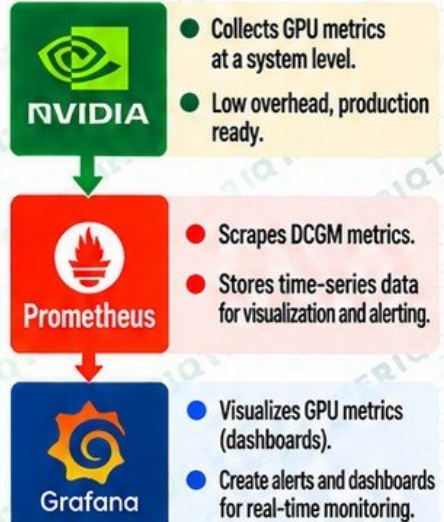
1. WHY MONITOR GPUS?

- Ensure optimal GPU utilization and performance.
- Detect bottlenecks and degradation early.
- Monitor temperature, power and memory usage.
- Prevent hardware failures and downtime.
- Plan capacity and control costs.
- Essential for production AI/ML, inference and HPC workloads.

2. KEY GPU METRICS TO MONITOR

	GPU Utilization	Percentage of GPU in use (SM utilization).
	Memory Usage	Total, used and free GPU memory (VRAM).
	Temperature	GPU core and memory temperature (°C).
	Power Usage	Power consumption (Watts).
	Clocks	Graphics, memory and SM clocks.
	Processes	Which applications are using the GPU.
	Health Status	Errors, throttling, ECC errors and device health.

3. MONITORING STACK OVERVIEW



4. CHECK GPU METRICS WITH NVIDIA-SMI

```
$ nvidia-smi
```

NVIDIA-SMI 550.54.14		Driver Version: 550.54.14		CUDA Version: 12.4	
GPU	Name	Persistence-M	Bus-Id	Memory-Usage	
0	NVIDIA RTX 6000	On	00000000:81:00:0 Off	1024MiB / 49140MiB	

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
	Id	Id				
0	N/A	N/A	1234	C	python	800MiB
0	N/A	N/A	5678	C	/usr/bin/Xorg	120MiB

- Shows GPU utilization, memory usage, temperature, power and running processes.
- Useful for quick checks and basic troubleshooting.
- For production monitoring, use DCGM with Prometheus and Grafana.

5. NVIDIA DCGM (DATA CENTER GPU MANAGER)



NVIDIA DCGM

- Collects detailed GPU metrics (utilization, memory, power, temperature, errors).
- Designed for data center and production environments.
- Provides exporters for Prometheus: dcm-exporter.
- Supports monitoring multiple GPUs across nodes.

```
# Install DCGM (Ubuntu)
sudo apt update
sudo apt install -y nvidia-dcgm

# Run DCGM exporter (Docker)
docker run -d --gpus all \
--name dcm-exporter -p 9400:9400 \
nvc.io/nvidia/k8s/dcm-exporter:latest
```

6. PROMETHEUS METRICS EXAMPLE

```
# GPU utilization
DCGM_FI_DEV_GPU_UTIL 75

# GPU memory usage (bytes)
DCGM_FI_DEV_FB_USED 5368709120

# GPU temperature (Celsius)
DCGM_FI_DEV_GPU_TEMP 62

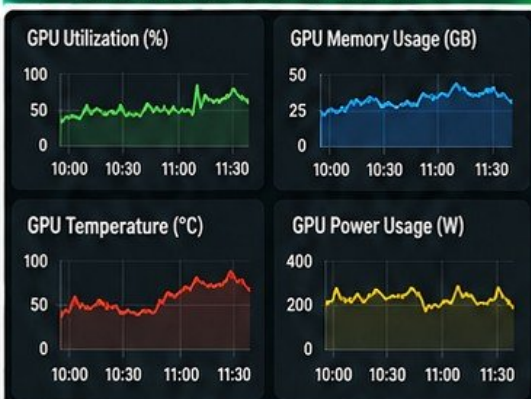
# GPU power usage (Watts)
DCGM_FI_DEV_POWER_USAGE 210.5

# GPU memory total (bytes)
DCGM_FI_DEV_FB_TOTAL 49152000000

# GPU SM clock (MHz)
DCGM_FI_DEV_SM_CLOCK 1410
```

- Metrics are exposed by dcm-exporter on port 9400.
- Prometheus scrapes these metrics.
- Use Grafana to visualize and create alerts (e.g., high temperature, low utilization, high memory usage).

7. GRAFANA DASHBOARD (EXAMPLE)



8. COMMON ALERTS

-  **High Temperature**
e.g., GPU_TEMP > 80°C
-  **High Power Usage**
e.g., POWER_USAGE > 300W
-  **High Memory Usage**
e.g., GPU memory > 90%
-  **Low Utilization**
e.g., utilization < 10% for long period
-  **GPU Errors**
ECC errors, Xid errors, device failures
-  **Unexpected Process**
Unknown or unauthorized GPU process

9. BEST PRACTICES

- Monitor utilization, memory, temperature and power.
- Use DCGM with Prometheus for production monitoring.
- Set up alerts for critical thresholds.
- Monitor multiple GPUs and nodes.
- Track trends to plan capacity.
- Combine GPU metrics with system metrics (CPU, memory, network).
- Regularly check for errors and hardware health.

10. KEY TAKEAWAYS

- Monitoring GPUs is essential for performance, reliability and cost control.
- Use nvidia-smi for quick checks.
- Use NVIDIA DCGM + Prometheus for production monitoring.
- Visualize metrics in Grafana.
- Set up alerts for temperature, power, memory and errors.
- Proper monitoring ensures stable, efficient and high-performing GPU workloads.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

18 GPU Troubleshooting 101

Diagnosing missing GPUs, driver failures, CUDA mismatches, memory exhaustion and unavailable resources.

1. COMMON GPU ISSUES



GPU not detected or missing.



Driver installation or driver failures.



CUDA version mismatch.



GPU memory exhaustion (out of memory).



GPU unavailable in container or Kubernetes.



Low GPU utilization or poor performance.



Overheating, high power usage or throttling.



Insufficient GPU resources or scheduling issues.

2. TROUBLESHOOTING FLOW

Identify the issue
(error message or symptom)

Check GPU detection
(nvidia-smi)

Verify driver and CUDA version
compatibility

Check GPU utilization,
memory, temperature and power

Check container / Kubernetes
configuration

Review logs and system events

Apply fix and verify
GPU is working

3. USEFUL DIAGNOSTIC COMMANDS

```
# Check GPU status
nvidia-smi # Show GPU information

# Check detailed info
nvidia-smi -q # Detailed GPU info

# Check running processes
nvidia-smi pmon # GPU processes

# Check driver version
nvidia-smi --query-gpu=driver_version # Driver version

# Check CUDA version (if installed)
nvcc --version # CUDA toolkit version

# Check system logs
sudo journalctl -k | grep -i nvidia # Kernel/driver logs

# Check GPU in Docker container
docker run --rm --gpus all nvidia/cuda:12.4.1-base
ubuntu nvidia-smi

# Check GPU resources in Kubernetes
kubectl describe node <node-name> | grep -i nvidia
kubectl get pods -o wide | grep -i gpu
```

4. GPU NOT DETECTED



- Check if the GPU is physically connected and enabled in BIOS/virtualization.
- Verify NVIDIA drivers are installed.
- Run `nvidia-smi` to confirm detection.
- Check PCIe devices: `lspci | grep -i nvidia`
- Ensure correct instance type (cloud).

5. DRIVER FAILURES



- Check driver version: `nvidia-smi`
- Reinstall or upgrade the NVIDIA driver.
- Check kernel compatibility.
- Review logs: `journalctl -k | grep nvidia`
- Ensure no conflicting drivers are installed.

6. CUDA MISMATCH



- Check CUDA version: `nvcc --version`
- Ensure driver supports the CUDA version.
- Use compatible CUDA container images.
- Match CUDA, driver and application versions.
- If using containers, pull the correct `nvidia/cuda` image tag.

7. GPU MEMORY EXHAUSTION



- Check memory usage: `nvidia-smi`
- Reduce batch size or model size.
- Clear unused processes.
- Use mixed precision or optimization techniques.
- Consider using MIG or multiple GPUs.

8. GPU UNAVAILABLE IN CONTAINERS



- Ensure NVIDIA Container Toolkit is installed.
- Run container with `--gpus all` flag.
- Use NVIDIA base images (`nvidia/cuda`).
- Verify container runtime configuration.
- Check GPU access inside the container: `nvidia-smi`

9. GPU UNAVAILABLE IN KUBERNETES



- Verify NVIDIA device plugin is installed.
- Check node labels and GPU capacity.
- Ensure correct resource requests: `resources.limits.nvidia.com/gpu: 1`
- Check pod events: `kubectl describe pod <pod>`
- Ensure scheduler can find a node with available GPUs.

10. OVERHEATING OR THROTTLING



- Check temperature: `nvidia-smi`
- Ensure proper cooling and airflow.
- Monitor power usage.
- Check for throttling: `nvidia-smi -q`
- Clean dust and verify data center environment.

11. INSUFFICIENT RESOURCES



- Check available GPUs: `nvidia-smi -L`
- Monitor cluster/node capacity.
- Use resource quotas and limits.
- Right-size instance type for workload.
- Consider using GPU autoscaling (cloud/K8s).

12. KEY TAKEAWAYS

- ✓ Start with `nvidia-smi` to confirm basic detection.
- ✓ Always verify driver and CUDA compatibility.
- ✓ Check logs for detailed error information.
- ✓ Use correct container and Kubernetes configuration.
- ✓ Monitor memory, temperature and power.
- ✓ Apply the right fix based on the specific issue.
- ✓ Proper monitoring and alerting prevent future problems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

19 GPU Security and Cost

Access control, container permissions, GPU resource quotas, idle capacity and cloud cost awareness.

1. GPU SECURITY OVERVIEW

- Protect GPU infrastructure, data and workloads.
- Control who can access GPUs.
- Secure containers and host configurations.
- Prevent unauthorized usage and data leakage.
- Reduce risk of abuse and unexpected costs.
- Use least privilege and proper monitoring.

2. ACCESS CONTROL (USERS AND ROLES)

- Use IAM roles and policies (cloud).
- Restrict who can launch GPU instances.
- Limit access to specific instance types.
- Use RBAC in Kubernetes.
- Allow only authorized users and teams.



3. CONTAINER PERMISSIONS

- Use NVIDIA Container Toolkit for GPU access.
- Run containers with minimal required permissions.
- Avoid running containers as root.
- Use read-only filesystems where possible.
- Limit device access to only GPU devices.
- Scan container images for vulnerabilities.

Run a GPU container with limited permissions

```
docker run --gpus all \
--user 1000:1000 \
--read-only \
--security-opt no-new-privileges \
nvcv.io/nvidia/pytorch:latest
```

4. GPU RESOURCE QUOTAS

- Use Kubernetes resource requests and limits.
- Set namespace-level quotas for GPUs.
- Prevent teams from consuming all GPUs.
- Ensure fair allocation across workloads.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: gpu-quota
  namespace: ai-team
spec:
  hard:
    nvidia.com/gpu: "4"
```



Limit GPU usage per namespace

5. IDLE GPU CAPACITY

- Monitor GPU utilization.
- Identify idle or underutilized GPUs.
- Use auto-scaling and scheduling policies.
- Shut down or scale in unused instances.
- Consider time-slicing or MIG for better utilization.

Idle GPU
(0 - 5% utilization)



Busy GPU
(70 - 100% utilization)



6. CLOUD COST AWARENESS

- Understand GPU instance pricing (e.g., AWS, Azure, GCP).
- Track usage and costs.
- Use spot instances for non-critical workloads.
- Set budgets and alerts.
- Optimize storage and data transfer costs.
- Stop idle instances and environments.



7. GPU ACCESS IN KUBERNETES (RBAC + QUOTAS)



- Use RBAC to control who can create GPU workloads.
- Set resource quotas to limit GPU usage per namespace.
- Use node selectors or taints to restrict where GPU pods run.

8. COST OPTIMIZATION STRATEGIES

- Right-size GPU instances.
- Use auto-scaling and scheduled shutdowns.
- Use spot instances for non-critical jobs.
- Optimize storage and data transfer.
- Monitor usage and remove idle resources.

9. BEST PRACTICES

- Enforce least privilege access.
- Use IAM, RBAC and network security controls.
- Secure containers and host configurations.
- Set GPU resource quotas and limits.
- Monitor GPU utilization and costs.
- Remove idle resources.
- Use cost alerts and budgets.
- Regularly review access and permissions.

10. COMMON RISKS

- Unauthorized access to GPUs.
- Containers with excessive permissions.
- No resource limits (GPU hogging).
- Idle GPUs increasing cloud costs.
- Unmonitored usage and unexpected bills.
- Data leakage from shared GPU environments.
- Insecure container images.
- Lack of audit logs and monitoring.

11. KEY TAKEAWAYS

- Secure GPU access with IAM and RBAC.
- Use minimal container permissions.
- Set resource quotas and limits.
- Monitor GPU utilization and idle capacity.
- Be aware of cloud costs and use optimization strategies.
- Remove unused resources.
- Proper security and cost management improves performance, reduces risk and saves money.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

20 Your First GPU Infrastructure Lab

A guided beginner exercise to inspect a GPU, run a containerized workload, observe resource usage and verify results.

1. LAB OBJECTIVES



- Inspect the GPU on your system.



- Run a containerized workload using the GPU.



- Observe GPU resource usage (utilization, memory, temperature).



- Verify the results.



- Understand basic troubleshooting steps if something does not work.

2. PREREQUISITES

- A Linux server or VM with an NVIDIA GPU (with drivers installed).
- NVIDIA drivers (verified with `nvidia-smi`).
- Docker installed.
- NVIDIA Container Toolkit installed.
- Internet access to pull container images.



3. CHECK GPU AVAILABILITY

Check GPU information
`nvidia-smi`

NVIDIA-SMI 550.54.14		Driver Version: 550.54.14		CUDA Version: 12.4	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC
0	NVIDIA RTX 6000	On	00000000:81:00:0	Off	0

Processes:						
GPU	GI ID	CI ID	PID	Type	Process name	GPU Memory Usage
0	N/A	N/A	1234	C	python	1024MiB

4. VERIFY DRIVER AND CUDA VERSION

```
# Check NVIDIA driver version
nvidia-smi --query-gpu=driver_version --format=csv

# Check CUDA version (if nvcc is installed)
nvcc --version
```

Example Output:

NVIDIA-SMI 550.54.14

Cuda compilation tools, release 12.4,
V12.4.131

5. RUN A CONTAINERIZED GPU WORKLOAD

```
# Pull a CUDA-enabled container image
docker pull nvidia/cuda:12.4.0-runtime-ubuntu22.04

# Run the container with GPU access
docker run --rm --gpus all \
  nvidia/cuda:12.4.0-runtime-ubuntu22.04 \
  nvidia-smi
```

- This runs a container with access to all GPUs (`--gpus all`).
- The container executes `nvidia-smi` to verify GPU visibility.
- You can replace `nvidia-smi` with your own application (e.g., a deep learning model).

6. OBSERVE GPU USAGE IN REAL TIME

Monitor GPU usage (updates every 2 seconds)
`nvidia-smi -l 2`

Metric	What to Look For
 GPU Utilization	Percentage of GPU in use (0% - 100%).
 Memory Usage	Used vs total GPU memory (VRAM).
 Temperature	Normal range: 30°C - 85°C (depends on GPU model).
 Power Usage	Current power draw (Watts).
 Processes	Which processes are using the GPU.

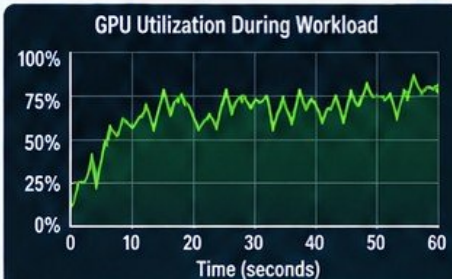
7. RUN A SAMPLE WORKLOAD (PYTORCH)

```
# Run a simple PyTorch GPU test
docker run --rm --gpus all \
  pytorch/pytorch:2.4.0-cuda12.4-cudnn9-runtime \
  python -c "import torch; \
  print('CUDA available:'); torch.cuda.is_available()); \
  print('GPU name:', torch.cuda.get_device_name(0)); \
  print('GPU memory (GB):', \
  round(torch.cuda.get_device_properties(0).total_memory/1e9,2))"
```

• Example Output:
CUDA available: True
GPU name: NVIDIA RTX 6000
GPU memory (GB): 48.0

8. OBSERVE RESOURCE USAGE DURING WORKLOAD

In another terminal, monitor GPU usage
`nvidia-smi -l 2`



9. VERIFY RESULTS

- ✓ GPU is visible inside the container.
- ✓ Application runs successfully using the GPU.
- ✓ GPU utilization increases during the workload.
- ✓ GPU memory usage is shown correctly.
- ✓ No errors in logs or container output.

10. CLEAN UP

```
# No cleanup needed for --rm containers

# To remove pulled images (optional)
docker rmi nvidia/cuda:12.4.0-runtime-ubuntu22.04
docker rmi pytorch/pytorch:2.4.0-cuda12.4-cudnn9-runtime
```

11. COMMON ISSUES

- ⚠ GPU not detected inside container.
- ⚠ Driver or CUDA version mismatch.
- ⚠ Permission denied (container access).
- ⚠ Out of GPU memory (OOM).
- ⚠ Low GPU utilization.
- ⚠ Container runtime not configured correctly.

12. KEY TAKEAWAYS

- ✓ You can inspect and verify GPU availability.
- ✓ You can run containerized workloads using the GPU.
- ✓ You can monitor real-time GPU resource usage.
- ✓ You can verify the results and troubleshoot issues.
- ✓ This forms the foundation for more advanced GPU infrastructure setups.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta